BAZE DE DATE

CURS 7

ALGEBRA RELAȚIONALĂ

- **LDD** – precizează **entităţile**, relaţiile dintre ele, **atributele**, structura atributelor, **cheile**, **constrângerile**, prin urmare defineşte **structura obiectelor bazei de date** (**schema** bazei de date)*;*

- **LMD** – cuprinde aspecte referitoare la introducerea, modificarea, eliminarea şi căutarea datelor*;*

ALGEBRA RELAȚIONALĂ

- Modelul relaţional oferă **două mulţimi de operatori pe relaţii**:
  - **algebra relaţională** (filtrele se obţin aplicând operatori specializaţi asupra uneia sau mai multor relaţii din cadrul bazei relaţionale);
  - **calculul relaţional** (filtrele se obţin cu ajutorul unor formule logice pe care tuplurile rezultatului trebuie să le satisfacă);

- Echivalenţa dintre algebra relaţională şi calculul relaţional a fost demonstrată de Jeffrey David Ullman. Această echivalenţă arată că orice relaţie posibil de definit în algebra relaţională poate fi definită şi în cadrul calculului relaţional, şi reciproc.

1. Algebra relațională

- Este o metodă **procedurală** – adică descrie **cum** se obține rezultatul. Se folosesc **operatori** pentru a manipula relațiile (tabelele).

2. Calculul relațional

- Este o metodă care descrie **ce** condiții trebuie să îndeplinească rezultatul, fără să specifice cum se obține.

Diferența principală:

- **Algebra relațională** = **cum** ajungem la rezultat, folosind operatori specifici;
- **Calcul relațional** = **ce** condiții trebuie să respecte datele rezultate, exprimat logic;

STUDENȚI

id#	nume	id_facultate
1	Andrei	10
2	Maria	20
3	Ioana	10

FACULTĂȚI

id_facultate#	nume_facultate
10	Informatica
20	Litere

Să se afișeze **numele studenților** care sunt la **Facultatea de Informatică**.

1. **Algebra relațională:**

R1 = JOIN (STUDENȚI, FACULTĂȚI, STUDENȚI.id_facultate = FACULTĂȚI.id_facultate)
R2 = SELECT (R1, nume_facultate = 'Informatica')
Rezultat = PROJECT (R2, nume);

2. Calculul relațional:

{ s.nume |
s ∈ Studenti AND
f ∈ Facultati AND
s.id_facultate = f.id_facultate AND
f.nume_facultate = 'Informatica'
}

STUDENȚI

id#	nume	id_facultate
1	Andrei	10
2	Maria	20
3	Ioana	10

FACULTĂȚI

id_facultate#	nume_facultate
10	Informatica
20	Litere

Să se afișeze **numele studenților** care sunt la **Facultatea de Informatică**.

1. **Algebra relațională:**

R1 = JOIN (STUDENȚI, FACULTĂȚI, STUDENȚI.id_facultate = FACULTĂȚI.id_facultate)
R2 = SELECT (R1, nume_facultate = 'Informatica')
Rezultat = PROJECT (R2, nume);

QUERY SQL:

```
SELECT s.nume
FROM Studenti s
JOIN Facultati f ON s.id_facultate = f.id_facultate
WHERE f.nume_facultate = 'Informatica';
```

Concluzie:

- **Algebra relațională –** procedurală, bazată pe operatori matematici
- **Calcul relațional –** logic, pe bază de condiții
- **SQL –** limbaj practic, inspirat din ambele, ușor de folosit de oameni

ALGEBRA RELAȚIONALĂ

- **Algebra relațională** a fost introdusă de ***Edgar Frank Codd*** și are la bază o **mulțime de operații** care se aplică asupra unor relații, rezultatul fiind tot o relație;

- Este un limbaj abstract cu ajutorul căruia se pot scrie cererile (se poate face o paralelă cu pseudocodul care se transformă în cod interpretat de mașină);

- Pentru a scrie cereri folosind ***algebra relațională*** avem nevoie de simboluri specifice, operații și implicit operatori;

- Cu alte cuvinte, în **algebra relațională** folosim anumiți operatori specifici, reprezentați cu ajutorul unor simboluri și notații, pe care ii aplicăm unor relații, rezultatul final fiind tot o relație;

ALGEBRA RELAȚIONALĂ

De exemplu:

Fie *R* si *S* două **relații** asupra cărora se aplică operatorul **UNION** (reuniune). Rezultatul reuniunii celor două relații este tot o relație și se notează:

Rezultat = UNION(R,S)

Prin algebra relațională se pot construi cererile pas cu pas până la obținerea unui rezultat final, prin scrierea ***expresiilor relaționale***.

ALGEBRA RELAȚIONALĂ

Scopul fundamental al algebrei relaționale este de a permite scrierea **expresiilor relaționale**

- acestea sunt o **reprezentare** de nivel superior, simbolică, a intenţiilor utilizatorului şi pot fi supuse unei diversităţi de **reguli de transformare** (**optimizare**).

Relaţiile sunt închise faţă de algebra relaţională

- operanzii şi rezultatele sunt relaţii → ieşirea unei operaţii poate deveni intrare pentru alta → posibilitatea imbricării expresiilor în algebra relaţională.

ALGEBRA RELAȚIONALĂ

Operatorii algebrei relaționale:

- operatori clasici pe mulțimi (**UNION**, **INTERSECT**, **PRODUCT**, **DIFFERENCE**)
- operatori relaționali speciali (**PROJECT**, **SELECT**, **JOIN**, **DIVISION**)

OPERATORUL PROJECT

- Operatorul **PROJECT** – se utilizează atunci când se dorește preluarea anumitor valori din baza de date (atributele/coloanele A1, A2, … , An), fără a fi necesară o condiție.
- Proiecţia este o operaţie unară care elimină anumite atribute ale unei relaţii producând o submulţime „pe verticală“ a acesteia.
  - Suprimarea unor atribute poate avea ca efect apariţia unor tupluri duplicate, care trebuie eliminate.
- **Notaţii**:
  - **PROJECT (R, A_1, ..., A_m)**
  - $\Pi_{A1, \ldots, Am}(R)$
  - $R[A_1, \ldots, A_m]$

unde $A_1, A_2, \ldots, A_m$ sunt parametrii proiecţiei relativ la relaţia R.

OPERATORUL PROJECT

Notație: ***PROJECT(R, A1, A2, … , An)***

Exemplu : Să se obțină o listă care cuprinde numele și job-ul angajaților.

Cerere SQL:

```
SELECT last_name, job_id
FROM employees;
```

Algebra relațională (expresia algebrică):

Rezultat = PROJECT(EMPLOYEES, nume, job);

OPERATORUL PROJECT

1. Proiecţie cu dubluri în SQL:

```
SELECT department_id
FROM employees;
```

2. Proiecţie fără dubluri în SQL:

```
SELECT DISTINCT department_id
FROM employees;
```

OPERATORUL SELECT

- Operatorul **SELECT** – se utilizează atunci când preluăm date pe baza unei condiții.
- Selecţia (restricția) este o operaţie unară care produce o submulţime pe „orizontală“ a unei relaţii *R.*
  - Această submulţime se obţine prin extragerea tuplurilor din *R* care **satisfac o condiţie specificată**.
- **Notaţii**:
  - **SELECT(*R*, condiţie)**
  - $\sigma_{\text{condiţie}}(R)$
  - *R*[condiţie]
  - RESTRICT(*R*, condiţie).

OPERATORUL SELECT

Notatie: ***SELECT(R, conditie)***

Exemplu : Să se obțină toate informațiile despre angajații care au jobul ‘IT_PROG’.

Cerere SQL:

```
SELECT *
FROM employees
WHERE job_id = 'IT_PROG';
```

Algebra relațională (expresia algebrică):

Rezultat = SELECT(EMPLOYEES, job_id = ‘IT_PROG’)

OPERATORUL UNION

- Operatorul **UNION** – reuniunea a două relații *R* și *S* (elementele comune și necomune afișate o singură dată).
- **Reuniunea** a două relaţii *R* şi *S* este mulţimea tuplurilor aparţinând fie lui *R*, fie lui *S*, fie ambelor relaţii.
- **Notaţii**:
  - **UNION(*R*, *S*)**
  - $R \cup S$
  - OR(*R*, *S*)
  - APPEND(*R*, *S*)

OPERATORUL UNION

Notație: ***UNION(R, S)***

Exemplu: Se cer codurile departamentelor al căror nume conține șirul “re” sau în care lucrează angajați având codul job-ului “SA_REP”.

Cerere SQL:

```
SELECT department_id
FROM departments
WHERE lower(department_name) like '%re%'

UNION

SELECT department_id
FROM employees
WHERE upper(job_id) like '%SA_REP%';
```

OPERATORUL UNION

```
SELECT department_id
FROM departments
WHERE lower(department_name) like '%re%'

UNION

SELECT department_id
FROM employees
WHERE upper(job_id) like '%SA_REP%';
```

Algebra relațională (expresia algebrică):

R1 = SELECT(DEPARTMENTS, department_id LIKE '%re%')
R2 = PROJECT(R1, department_id)
R3 = SELECT(EMPLOYEES, job_id LIKE '%SA_REP%')
R4 = PROJECT(R3, department_id)
***Rezultat** = UNION(R2, R4)*

OPERATORUL UNION

Algebra relationala (expresia algebrica):

R1 = SELECT(DEPARTMENTS, department_id LIKE '%re%')
R2 = PROJECT(R1, department_id)
R3 = SELECT(EMPLOYEES, job_id LIKE '%SA_REP%')
R4 = PROJECT(R3, department_id)
***Rezultat** = UNION(R2, R4)*

Explain Plan

SQL | 0.006 seconds

OPERATION	OBJECT_NAME	OPTIONS
SELECT STATEMENT		
HASH		UNIQUE
UNION-ALL		
TABLE ACCESS	DEPARTMENTS	FULL
Filter Predicates		
LOWER(DEPARTMENT_NAME) LIKE '%re%'		
TABLE ACCESS	EMPLOYEES	FULL
Filter Predicates		
UPPER(JOB_ID) LIKE '%SA_REP%'		

OPERATORUL DIFFERENCE

- Operatorul **DIFFERENCE** – diferența a două relații *R* și *S* (elementele care sunt în R și nu sunt în S) – este mulţimea tuplurilor care aparţin lui *R*, dar nu aparţin lui *S*.
- Diferenţa este o operaţie binară necomutativă care permite obţinerea tuplurilor ce apar numai într-o relaţie.
- **Notaţii**:
  - **DIFFERENCE(*R*, *S*)**
  - R – S
  - REMOVE(*R*, *S*)
  - MINUS(R, S)

OPERATORUL DIFFERENCE

Notație: ***DIFFERENCE(R, S)***

Exemplu: Să se obțină codurile departamentelor în care nu lucrează nimeni.

Cerere SQL:

SELECT department_id
FROM departments -- department_id este cheie primară, deci avem o listă
-- unică de departamente

MINUS -- din lista tuturor departamentelor dorim să eliminam
-- departamentele în care lucrează angajați

SELECT department_id
FROM employees; -- department_id este cheie externă, deci sunt
-- departamente în care lucrează angajați

Algebra relațională (expresia algebrică):

R1 = PROJECT(DEPARTMENTS, department_id)
R2 = PROJECT(EMPLOYEES, department_id)
Rezultat = DIFFERENCE(R1, R2)

OPERATORUL INTERSECT

- Operatorul **INTERSECT** – intersecția a două relații *R* și *S* (elementele comune celor două relații afișate o singură dată).
- Intersecţia a două relaţii *R* şi *S* este mulţimea tuplurilor care aparţin atât lui *R*, cât şi lui *S*. Operatorul INTERSECT este un operator binar, comutativ, derivat:
  - $R \cap S = R - (R - S)$
  - $R \cap S = S - (S - R)$.
- **Notaţii**:
  - **INTERSECT(R, S)**
  - $R \cap S$
  - AND(R, S)
- Operatorii **INTERSECT** şi **DIFFERENCE** pot fi simulaţi în SQL (în cadrul comenzii SELECT) cu ajutorul opţiunilor EXISTS, NOT EXISTS, IN, NOT IN (!= ANY) – o să implementam în laborator toate exemplele

OPERATORUL INTERSECT

Notație: ***INTERSECT(R, S)***

Exemplu: Să se afișeze angajații (codul lor) care sunt manageri în cadrul departamentelor și în același timp sunt și managerii altor angajați.

Cerere SQL:

SELECT manager_id
FROM departments -- managerii de departament

INTERSECT

SELECT manager_id
FROM employees; -- managerii angajaților

Algebra relațională (expresia algebrică):

R1 = PROJECT(DEPARTMENTS, manager_id)
R2 = PROJECT(EMPLOYEES, manager_id)
Rezultat = INTERSECT(R1, R2)

OPERATORUL INTERSECT

Intersecția se poate simula utilizând operatorul **IN** sau operatorul **EXISTS (îl vom studia în laborator).**

Cum procedăm atunci când utilizăm operatorul IN?

Cerere SQL (simularea intersecției folosind operatorul **IN**):

```
SELECT manager_id
FROM departments
WHERE manager_id IN (SELECT manager_id
                     FROM employees ) ;
```

OPERATORUL PRODUCT

- Operatorul **PRODUCT** – produsul cartezian dintre relația *R* și relația *S*.
- Fie *R* şi *S* relaţii de aritate ***m***, respectiv ***n***. Produsul cartezian al lui *R* cu *S* este mulţimea tuplurilor de aritate ***m* + *n*** unde primele *m* componente formează un tuplu în *R*, iar ultimele *n* componente formează un tuplu în *S*.
- **Notaţii:**
  - **PRODUCT(R, S)**
  - $R \times S$
  - TIMES(R, S)

OPERATORUL PRODUCT

Notație: ***PRODUCT(R, S)***

Exemplu: Să se afișeze toți angajații (codurile și numele angajaților) împreună cu toate departamentele (numele și codurile departamentelor).

Cerere SQL:

SELECT employee_id, last_name, d.department_id, department_name
FROM employees e, departments d;

Algebra relațională (expresia algebrică):

R1 = PROJECT(EMPLOYEES, employee_id, last_name)
R2 = PROJECT(DEPARTMENTS, department_id, department_name)
Rezultat = PRODUCT(R1, R2)

Cerere SQL (utilizând CROSS JOIN):

SELECT employee_id, last_name, d.department_id, department_name
FROM employees e CROSS JOIN departments d;

OPERATORUL JOIN

- Operatorul **JOIN** – permite regăsirea informației din mai multe relații corelate. Operatorul combină **produsul cartezian** cu **selecția** și **proiecția**.

- 4 tipuri de **JOIN**:
  - **NATURAL JOIN**
  - **θ-JOIN**
  - **SEMI-JOIN**
  - **OUTER JOIN**

NATURAL JOIN

- **NATURAL JOIN** – operatorul de compunere naturală combină tupluri din două relaţii *R* şi *S*, cu condiţia ca **atributele comune să aibă valori identice**.

- Cu alte cuvinte, acest tip de join realizează join automat bazat pe coloanele comune existente în cele două tabele între care se realizează operația de join. Coloanele comune sunt cele care au același nume în ambele tabele.

NATURAL JOIN

Algoritmul care realizează compunerea naturală este următorul:

1. se calculează produsul cartezian ***R*** $\times$ ***S***;
2. pentru fiecare atribut comun ***A*** care defineşte o coloană în *R* şi o coloană în *S*, se selectează din *R* $\times$ *S* tuplurile ale căror valori coincid în coloanele ***R.A*** şi ***S.A*** (atributul *R.A* reprezintă numele coloanei din *R* $\times$ *S* corespunzătoare coloanei *A* din *R*);
3. pentru fiecare astfel de atribut *A* se elimină coloana *S.A*, iar coloana *R.A* se va numi *A*.

NATURAL JOIN

- Operatorul NATURAL JOIN poate fi exprimat formal astfel:

$$\mathbf{JOIN(R, S) = \Pi_{i1,\ldots,im}\ \sigma_{(R.A1 = S.A1)\wedge \ldots \wedge(R.Ak = S.Ak)}(R \times S)},$$

unde $A_1, \ldots, A_k$ sunt atributele comune lui R şi S, iar $i_1, \ldots, i_m$ reprezintă lista componentelor din $R \times S$ (păstrând ordinea iniţială) din care au fost eliminate componentele $S.A_1, \ldots, S.A_k$.

NATURAL JOIN

Notație: ***JOIN(R, S)***

Exemplu: Să se afișeze informații despre angajații care lucrează în departamente și care sunt în același timp atât manageri de departament, cât și managerii altor angajati.

Algoritmul compunerii naturale:

1. Se calculează produsul cartezian dintre relațiile R și S (R reprezintă relația employees, iar S relația departments);

SELECT employee_id, last_name, d.department_id, department_name, d.manager_id
FROM ***employees e, departments d****;*

NATURAL JOIN

2. Se selectează tuplurile pentru care există valori comune atât în relația R, cât și în relația S (valorile comune sunt – department_id și manager_id – acestea făcând parte din ambele relații)

WHERE e.department_id = d.department_id AND
e.manager_id = d.manager_id;

3. Pentru fiecare atribut existent în ambele relații, în afișare se păstrează doar o singură dată coloana comună (în cazul nostru o să avem o singură dată coloana department_id și o singură dată coloana manager_id).

NATURAL JOIN

Algoritmul anterior se aplică automat atunci când utilizăm **NATURAL JOIN**, nefiind nevoie de aliasuri.

SELECT employee_id, last_name, department_id, department_name, manager_id
FROM employees ***NATURAL JOIN*** *departments;*

Algebra relațională (expresia algebrică):

Rezultat = JOIN(EMPLOYEES, DEPARTMENTS)

θ-JOIN

- **θ-JOIN** – combină tupluri (coloane/atribute) din două relații R și S pe baza unei condiții specificate în cadrul operației de JOIN. Această condiție poate fi bazată pe egalitatea unor coloane (de obicei cheia primară și cheia externă) – acesta fiind un caz particular de **θ-JOIN**, și anume **EQUIJOIN (sau inner join)**, dar sunt utilizate în egală măsură și condiții care implică operatorii <, <=, >, >=, != (acest tip de join fiind un **NONEQUIJOIN**)

- Operatorul **θ-JOIN** este un operator derivat, fiind o combinaţie de produs scalar şi selecţie:

$$\mathbf{JOIN}(R, S, \text{condiţie}) = \sigma_{\text{condiţie}}\,(R \times S)$$

θ-JOIN

Notație: ***JOIN(R, S, conditie)***

Exemplu: Să se afișeze numele angajaților împreună cu denumirea jobului în cadrul căruia lucrează.

Cerere SQL:

SELECT last_name, job_title
FROM employees e, jobs j
WHERE e.job_id = j.job_id;

Algebra relațională (expresia algebrică):

R1 = PROJECT(EMPLOYEES, last_name, job_id)
R2 = PROJECT(JOBS, job_title, job_id)
R3 = JOIN(R1, R2, EMPLOYEES.job_id = JOBS.job_id)
Rezultat = PROJECT(R3, last_name, job_title);

SEMI-JOIN

- **SEMI-JOIN** – se utilizează atunci când din cele două relații participante *R* și *S* se utilizează doar atributele relației R sau doar cele ale relației S.

- Operatorul SEMI-JOIN conservă **atributele unei singure relaţii participante** la compunere şi este utilizat când nu sunt necesare toate atributele compunerii. Operatorul este asimetric.
  - Tupluri ale relaţiei R care participă în compunerea (naturală sau θ-JOIN) dintre relaţiile R şi S.

- SEMI-JOIN este un operator derivat, fiind o combinaţie de proiecţie şi compunere naturală sau proiecţie şi θ-JOIN:

Notație: ***SEMIJOIN(R, S) = PROJECT(JOIN(R, S))***
SEMIJOIN(R, S, conditie) = PROJECT(JOIN(R, S, conditie))

SEMI-JOIN

Exemplu: Să se afișeze departamentele (codul și denumirea) care au cel puțin un angajat.

Se afișează în final coloane care fac parte doar dintr-un tabel – departments => operatorul SEMI-JOIN conservă **atributele unei singure relaţii participante**.

SELECT d.department_id, department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id;

Algebra relațională (expresia algebrică):

R1 = JOIN(EMPLOYEES, DEPARTMENTS, EMPLOYEES.department_id = DEPARTMENTS.department_id)
Rezultat = PROJECT(R1, department_id, department_name)

OUTER JOIN

- **OUTER JOIN** – pentru două relații R și S, acest operator returnează valorile atributelor care îndeplinesc condiția de corelare (adică intersecția celor două relații pe baza unei valori comune – join clasic), împreuna cu valori NULL în funcție de tipul acestui operator.
- Cu alte cuvinte, operaţia de **compunere externă** combină tupluri din două relaţii, tupluri pentru care nu sunt satisfăcute condiţiile de corelare.
- În cazul aplicării operatorului JOIN se pot pierde tupluri, atunci când **există un tuplu în una din relaţii pentru care nu există niciun tuplu în cealaltă relaţie**, astfel încât să fie satisfăcută relaţia de corelare.
- Operatorul elimină acest inconvenient prin **atribuirea valorii *null*** valorilor atributelor care există într-un tuplu din una dintre relaţiile de intrare, dar care nu există şi în cea de-a doua relaţie.
- Practic, se realizează compunerea a două relaţii *R* şi *S* la care se adaugă tupluri din *R* şi *S*, care nu sunt conţinute în compunere, completate cu valori *null* pentru atributele care lipsesc.

OUTER JOIN

- Compunerea externă poate fi: LEFT, RIGHT, FULL.

- **De exemplu:**

  R LEFT OUTER JOIN S – reprezintă compunerea în care tuplurile din *R*, care nu au valori similare în coloanele comune cu relaţia *S*, sunt de asemenea incluse în relaţia rezultat.

- Asadar,

  R **LEFT OUTER JOIN** S – se vor afișa valorile din *R* și *S* care respectă condiția de JOIN (intersecția) împreună cu valorile care sunt în *R* (LEFT) și nu sunt în *S*. În acest caz valorile care lipsesc vor fi automat completate cu NULL.

OUTER JOIN

Exemplu:

Considerăm tabelul **EMPLOYEES ca fiind relația R** și tabelul **DEPARTMENTS ca fiind relația S**

Să se afișeze angajații (cod și nume) împreună cu departamentele în care lucrează (cod și denumire). Rezultatul o să includă toți angajații chiar dacă nu au departament.

*SELECT employee_id, **d.**department_id, last_name, department_name FROM employees e **LEFT JOIN** departments d ON (e.department_id = d.department_id);*

OUTER JOIN

- În output se observă angajații care lucrează într-un departament, adică valorile din ***EMPLOYEES*** și ***DEPARTMENTS*** (din *R* și *S*) care îndeplinesc condiția de corelare (**e.department_id = d.department_id**), împreună cu valorile care sunt în *EMPLOYEES* (R) și nu sunt în *DEPARTMENTS* (S), adică angajații care nu au departament.

- Valorile care lipsesc vor fi completate automat cu **NULL** – în exemplul nostru valoarea care lipsește este cea specifică departamentului deoarece sunt afișați și angajații **care nu au departament,** ceea ce înseamnă că angajatul există, dar departamentul nu. În acest caz o să existe un rezultat ca acesta :

  178 NULL Grant NULL – coloanele reprezentative departamentului (department_id și department_name sunt automat completate cu valoarea NULL)

OUTER JOIN

- În mod asemanător se vor comporta și variantele următoare de **JOIN** (*RIGHT OUTER JOIN* și *FULL OUTER JOIN*)

- R **RIGHT OUTER JOIN** S – se vor afișa valorile din R și S care respectă condiția de JOIN (intersecția) împreună cu valorile care sunt în S (RIGHT) și nu sunt în R. În acest caz valorile care lipsesc vor fi automat completate cu NULL.

- R **FULL OUTER JOIN** S – se vor afișa valorile din R și S care respectă condiția de JOIN (intersecția) împreună cu valorile care sunt în R și nu sunt în S și valorile care sunt în S și nu sunt în R. În acest caz valorile care lipsesc vor fi automat completate cu NULL.

OPERATORUL DIVISION

- **Diviziunea** este o operaţie binară care defineşte o relaţie ce conţine valorile atributelor dintr-o relaţie care apar **în toate** valorile atributelor din cealaltă relaţie.
- **Notaţii**:
  - **DIVISION(R, S)**
  - DIVIDE(R, S)
  - $R \div S$
- Diviziunea conţine acele tupluri de dimensiune $n - m$ la care, adăugând orice tuplu din S, se obţine un tuplu din R.
- Operatorul diviziune poate fi exprimat formal astfel:
  - $\boldsymbol{R^{(n)} \div S^{(m)} = \{t^{(n-m)} \mid \forall\, s \in S, (t, s) \in R\}}$, unde $n > m$ şi $S \neq \emptyset$.

ALGEBRA RELAȚIONALĂ

- Operatorul **DIVISION** este legat de cuantificatorul universal (∀) care nu există în SQL.

- Cuantificatorul universal poate fi însă simulat cu ajutorul cuantificatorului existenţial (∃) utilizând relaţia:

$$\forall x\ P(x) \equiv \neg \exists\ x \neg P(x).$$

- Prin urmare, operatorul **DIVISION** poate fi exprimat în SQL prin succesiunea a doi operatori NOT EXISTS.

- **Acest operator o să fie studiat în cadrul laboratorului**